PROJECT PROPOSAL

Saliency-Aware Generative Neural Compression

An End-to-End Learned Image Compression System for Perceptually Optimized Bit Allocation

---

Research focus

Saliency-conditioned latent allocation, actual entropy-coded bitstreams, and perceptual image reconstruction.

Proposed duration: 10 weeks

Technical proposal and implementation roadmap

1. Executive Summary

This project proposes the development of an end-to-end neural image compression system that uses perceptual importance, particularly visual saliency, to improve image quality at constrained bitrates.

Traditional image compression methods generally optimize reconstruction quality using pixel-level distortion metrics. However, not all regions of an image are equally important to human perception. A small region containing a face, object, or other visually significant content may matter more than a large background region.

The proposed system will investigate whether **saliency-aware, spatially adaptive bit allocation** can preserve perceptually important image regions more effectively than conventional uniform compression at the same overall bitrate.

The system will consist of four principal components:

- **Saliency Estimation Module:** Identifies visually important regions in an input image.
- **Neural Encoder:** Converts the image into a compact latent representation.
- **Adaptive Compression and Entropy Coding:** Allocates representational capacity according to spatial importance and produces an actual compressed bitstream.
- **Generative Decoder:** Reconstructs the image from the compressed representation, with optional perceptual and adversarial training objectives.

The project will also establish a reproducible machine learning engineering workflow using GitHub, MLflow, DVC, automated testing, experiment configurations, and documented evaluation procedures.

The primary research contribution will be the investigation and implementation of saliency-conditioned latent allocation, rather than simply combining existing neural compression and generative modelling techniques.

2. Problem Statement

Conventional compression algorithms aim to reduce image size while maintaining acceptable reconstruction quality. However, pixel-level reconstruction objectives do not necessarily correspond to human perceptual quality.

- A compression algorithm may preserve background textures while degrading a person's face.
- Important objects may lose fine details at low bitrates.
- Uniform quantization may allocate compression capacity inefficiently across different image regions.
- Pixel-level metrics such as PSNR may not fully reflect perceptual differences between reconstructed images.

Central research problem: Can a learned image compression system use visual saliency to allocate its latent coding capacity more effectively, improving reconstruction quality in important regions without increasing the overall bitrate?

3. Research Objectives

- Develop a functional end-to-end neural image compression baseline.
- Implement a saliency estimation mechanism to identify perceptually important image regions.
- Investigate saliency-conditioned latent representations and adaptive bit allocation.
- Implement differentiable quantization and entropy modelling.
- Generate a decodable compressed bitstream and measure actual bitrate.
- Investigate perceptual reconstruction objectives using LPIPS and MS-SSIM.
- Explore adversarial training to improve perceptual reconstruction quality, subject to available computational resources.
- Compare the proposed system against conventional image codecs and established learned compression baselines.
- Conduct ablation studies to determine the contribution of individual components.

- Establish a reproducible experimentation and software development pipeline.

4. Proposed System Architecture

The proposed architecture follows an incremental end-to-end pipeline. A standard learned codec will be implemented first; saliency-conditioned allocation and generative enhancements will be introduced only after the baseline is validated.

Component	Function	Expected output
Input image	Accepts RGB image and performs preprocessing.	Normalized image tensor
Saliency estimation	Predicts visually important regions.	Spatial saliency map
Neural encoder	Transforms the image into a compact latent representation.	Latent tensor
Adaptive allocation	Conditions latent representation or quantization on importance.	Adaptively allocated latent representation
Quantization and entropy coding	Quantizes symbols and encodes them using a probability model.	Compressed bitstream
Generative decoder	Reconstructs the image from the decoded latent representation.	Reconstructed image
Evaluation	Measures rate, distortion, perceptual quality and ROI fidelity.	Metrics and benchmark reports

4.1 Saliency Estimation Module

The saliency module will estimate which image regions attract greater visual attention. The initial implementation will use a lightweight pretrained saliency prediction network.

- Generate a normalized spatial saliency map.
- Resize or downsample the map to align with the encoder's latent feature dimensions.
- Use the map to condition the latent representation or adaptive quantization mechanism.
- Begin with a frozen pretrained estimator to reduce training complexity; joint optimization may be explored later.

4.2 Neural Encoder

The encoder transforms the input image into a lower-dimensional latent representation. Candidate components include a convolutional neural network, residual blocks and downsampling layers. Attention mechanisms or Swin Transformer blocks may be explored in later experiments. The initial baseline should use an established learned image compression architecture.

4.3 Saliency-Conditioned Adaptive Allocation

This is the primary research component. The saliency map will influence how compression capacity is distributed across the latent representation.

- **Latent modulation:** Use saliency features to modify latent activations.
- **Adaptive quantization:** Apply spatially varying quantization behaviour based on importance.
- **Channel or spatial gating:** Control the amount of information retained in different regions.
- **Rate allocation:** Encourage the entropy model to assign more coding capacity to important regions.

The project will first establish a saliency-weighted distortion baseline, then compare it with a genuine adaptive allocation mechanism. Merely assigning larger distortion penalties to salient regions does not prove that more bits are allocated there.

4.4 Differentiable Quantization and Entropy Modelling

During training, differentiable approximations such as additive uniform noise may be used. During inference, latent representations must be quantized, symbol probabilities estimated, and symbols encoded using an entropy coder. The resulting bitstream must be independently decodable. Actual BPP must include all required side information, headers and metadata.

4.5 Generative Decoder

The decoder reconstructs the image using the compressed latent representation. It will initially prioritize stable reconstruction using convolutional decoder blocks and residual reconstruction layers. Perceptual feature supervision and a PatchGAN discriminator may be introduced after the baseline and adaptive allocation stages are stable.

5. Mathematical Formulation

The compression system will optimize a rate-distortion objective:

$$\mathbf{L = R + \lambda D_{perceptual} + \beta L_{adversarial}}$$

Here, R is the estimated rate cost, D is perceptual reconstruction distortion, $L_{adversarial}$ is the adversarial loss, and λ and β control the trade-off weights.

A saliency-weighted distortion term can be formulated as:

$$\mathbf{D_{saliency} = \Sigma_{i,j} S_{ij} \cdot d(x_{ij}, x\blacksquare_{ij}) / \Sigma_{i,j} S_{ij}}$$

S is the spatial importance map, x is the original image, x■ is the reconstruction, and d is a distortion function. An initial rate-distortion objective is $L_{RD} = R + \lambda D_{saliency}$. The final objective may combine pixel-level and perceptual terms based on experiments. These are proposed formulations; the final loss and rate estimator will be fixed after baseline implementation.

6. Dataset Strategy

Dataset	Intended purpose
DIV2K	Primary image training dataset
COCO	Additional validation and generalization testing
Kodak PhotoCD	Standardized compression evaluation

- Use DIV2K or a suitable image-centric dataset for training.
- Use COCO or another independent image set for validation and generalization.
- Reserve Kodak for final benchmark evaluation.
- Generate image patches during training to control memory usage.
- Maintain reproducible dataset splits and version dataset metadata, preprocessing configurations and split definitions using DVC.
- Generate saliency maps using a pretrained model or an appropriate saliency dataset; document their generation and preprocessing.

7. Baselines and Comparative Evaluation

Baseline	Purpose
JPEG	Conventional image compression baseline
WebP	Modern image compression baseline
AVIF	Modern image compression baseline
CompressAI factorized-prior model	Learned compression baseline
CompressAI hyperprior model	Stronger learned compression baseline
Saliency-weighted distortion only	Tests whether adaptive allocation adds value
Proposed saliency-conditioned codec	Main proposed method

H.264 is primarily a video codec and should not be treated as a directly equivalent still-image baseline unless a specific intra-frame comparison protocol is defined. Learned baselines should be evaluated under a consistent protocol and compared at matched bitrates.

8. Evaluation Metrics

Metric	Purpose
Bits Per Pixel (BPP)	Compression efficiency
PSNR	Pixel-level reconstruction quality
MS-SSIM	Structural similarity
LPIPS	Perceptual similarity
ROI PSNR / ROI LPIPS	Reconstruction quality in important regions
Background distortion	Quality outside important regions
Encoding / decoding time	Compression and reconstruction speed
Peak GPU memory	Computational requirements
Actual bitstream size	Real compression performance

Rate-distortion analysis

Generate rate-distortion curves comparing BPP against PSNR and MS-SSIM, with perceptual comparisons using LPIPS.

Region-specific evaluation

- Salient-region reconstruction quality.
- Background-region reconstruction quality.
- Difference between salient and non-salient distortion.
- Overall quality at matched bitrate.

ROI masks and evaluation rules must be fixed before final testing to avoid tuning the evaluation protocol around the results.

Ablation studies

- Standard learned codec without saliency.

- Codec with saliency-weighted distortion.
- Codec with saliency-conditioned latent modulation.
- Codec with adaptive quantization or rate allocation.
- Codec with perceptual loss.
- Codec with optional adversarial training.

9. Technology Stack

Area	Technology	Responsibility
Programming	Python 3.11+	Core implementation
Deep learning	PyTorch	Model development and training
Learned compression	CompressAI	Baseline codec and entropy models
Image processing	Pillow, OpenCV, torchvision	Loading and preprocessing
Numerical processing	NumPy	Array operations
Evaluation	TorchMetrics, scikit-image	PSNR, MS-SSIM and metrics
Perceptual evaluation	LPIPS	Perceptual similarity
Experiment tracking	MLflow	Runs, metrics, parameters and artifacts
Dataset versioning	DVC	Dataset and preprocessing versioning
Source control	Git and GitHub	Version control and collaboration
Configuration	YAML, Hydra	Reproducible experiment configuration
Testing	pytest	Unit and integration testing
Code quality	Ruff, pre-commit	Formatting and static checks
Optimization	Optuna	Optional later-stage tuning
Visualization	Matplotlib	Rate-distortion curves and analysis
Demo	Streamlit or FastAPI	Optional inference interface
Environment	Docker	Reproducible runtime

Keep the initial infrastructure lightweight. MLflow can begin with a local SQLite backend and filesystem artifact storage. PostgreSQL and MinIO can be introduced if shared remote tracking or centralized artifact storage becomes necessary. Kubernetes, distributed training and complex orchestration are not required initially.

10. MLOps and Experiment Management

10.1 MLflow experiment tracking

MLflow will track every meaningful model experiment.

- **Parameters:** architecture, learning rate, batch size, epochs, optimizer, scheduler, loss weights, quantization, latent dimensions, saliency and dataset settings.
- **Metrics:** training/validation loss, estimated and actual BPP, PSNR, MS-SSIM, LPIPS, ROI metrics, encoding/decoding time and peak GPU memory.
- **Artifacts:** checkpoints, plots, rate-distortion curves, reconstructed samples, saliency maps, allocation visualizations, evaluation reports and configurations.

- **Provenance:** Git commit hash, DVC dataset revision, experiment configuration, Python/library versions and hardware information.

Suggested experiment names:

```
baseline_factorized\nbaseline_hyperprior\nsaliency_weighted_rd\nsaliency_latent_modulation\nadaptive_quantization\nperceptual
```

A run is complete only when it has evaluation results and sufficient metadata to reproduce it.

10.2 DVC dataset and artifact versioning

Git stores source code, configuration files, documentation, tests, DVC metadata and pipeline definitions. DVC manages dataset files or references, validation/test data, manifests, preprocessed data, generated saliency maps where appropriate, and selected large intermediate artifacts. Model checkpoints and experiment outputs can be stored through MLflow; DVC can version selected checkpoints when needed. Do not commit large datasets or checkpoints directly to Git.

10.3 Reproducible training pipeline

Stage	Activities
1. Data preparation	Acquisition, validation, preprocessing and split generation
2. Baseline training	Train and validate the initial learned codec
3. Saliency integration	Generate saliency maps and train saliency-aware variants
4. Adaptive allocation	Implement and evaluate spatially adaptive coding
5. Final evaluation	Bitstream validation, benchmarks, ablations and reporting

A dvc.yaml pipeline can define data preparation and evaluation stages. Training should remain configurable so that experiments can be reproduced using recorded YAML configurations.

11. GitHub-Based Development and Project Management

GitHub will be the central platform for source control, development, code review, issue tracking and project documentation.

11.1 Proposed repository structure

```
saliency-aware-neural-compression/
■■■ configs/
■   ■■■ data/
■   ■■■ model/
■   ■■■ training/
■   ■■■ experiments/
■■■ src/
■   ■■■ data/
■   ■■■ models/
■   ■■■ codec/
■   ■■■ losses/
■   ■■■ training/
■   ■■■ evaluation/
■   ■■■ utils/
■■■ scripts/
■   ■■■ prepare_data.py
■   ■■■ train.py
■   ■■■ evaluate.py
■   ■■■ encode_image.py
■   ■■■ decode_image.py
■   ■■■ benchmark.py
■■■ tests/
■■■ app/
■■■ docs/
■■■ data/
■■■ checkpoints/
■■■ outputs/
■■■ dvc.yaml
■■■ dvc.lock
■■■ pyproject.toml
■■■ .gitignore
■■■ .pre-commit-config.yaml
■■■ README.md
■■■ LICENSE
```

The data, checkpoints and outputs directories are runtime locations and should not contain large tracked files in Git.

11.2 Branch-based development

Branch	Purpose
main	Stable project version
feature/saliency-module	New feature implementation
feature/adaptive-allocation	Adaptive coding mechanism
feature/entropy-coder	Entropy coding and bitstream
feature/evaluation-pipeline	Metrics and benchmarks
fix/bitstream-decoding	Bug fixes
experiment/perceptual-loss	Experimental implementation

Use a protected main branch containing stable, reviewed code. Feature branches should be short-lived and merged through pull requests.

11.3 Pull request workflow

- **Create an issue:** Define the task, expected result and acceptance criteria.
- **Create a feature branch:** Branch from the latest main.
- **Implement the change:** Follow agreed module interfaces and coding standards.
- **Run tests and checks:** Execute relevant unit tests, linting and CPU smoke tests.
- **Open a pull request:** Include issue reference, implementation summary, test results and relevant evidence.
- **Review and resolve feedback:** Check correctness, integration compatibility, reproducibility and documentation.

- **Merge into main:** Only after required checks and review are complete.

11.4 Recommended GitHub settings

- Protect main against direct pushes.
- Require at least one approving review.
- Require successful CI checks.
- Require branches to be up to date before merging where practical.
- Enable automatic deletion of merged branches.
- Use squash merging for a clean history.

11.5 Commit conventions

```
feat: implement saliency estimation module
feat: add adaptive latent modulation
feat: implement entropy coding
fix: resolve decoder shape mismatch
test: add bitstream roundtrip tests
refactor: separate rate and distortion losses
docs: update architecture documentation
exp: add lpips training configuration
```

11.6 GitHub Issues and Projects

Status	Meaning
Backlog	Planned but not ready
Ready	Defined and ready for implementation
In Progress	Currently being developed
In Review	Pull request opened
Testing	Integration or validation underway
Done	Acceptance criteria satisfied
Blocked	Waiting on a dependency or decision

Recommended labels: type:feature, type:bug, type:research, type:documentation, type:experiment; priority:high/medium/low; area:codec/saliency/training/evaluation/mlops; compute:cpu/gpu.

11.7 Continuous integration

- Python syntax and import checks.
- Ruff linting.
- pytest unit tests.
- Model shape and tensor compatibility tests.
- CPU smoke tests.
- Configuration validation.
- Bitstream encode/decode round-trip tests where practical.

Full GPU training should not run on every pull request. GPU experiments should be executed separately and their results recorded in MLflow.

Definition of Done: A task is complete only when its implementation, relevant tests, documentation and pull request are merged, and any required experiment results are recorded.

12. Development Roadmap and Project Lifecycle

The project will follow an incremental development lifecycle. Each stage must produce a working, testable outcome before the next stage begins.

Phase	Duration	Main outcome
1. Research and setup	Week 1	Research specification and development environment
2. Dataset and baseline	Weeks 2–3	Working learned compression baseline
3. Saliency integration	Weeks 4–5	Saliency-aware distortion and latent conditioning
4. Adaptive allocation	Weeks 6–7	Main research contribution
5. Perceptual enhancement	Weeks 8–9	Perceptual objective and optional GAN extension
6. Final evaluation	Week 10	Final benchmarks, report and demonstration

Phase 1 — Research, design and infrastructure (Week 1)

- Review related learned image compression literature.
- Finalize the research question and evaluation protocol.
- Establish the GitHub repository, Python environment, MLflow and DVC.
- Define dataset structure and configuration conventions.
- Design initial model interfaces and establish issues, milestones and PR templates.

Deliverables: Research specification, literature review, architecture document, repository, MLflow/DVC setup and CI pipeline.

Phase 2 — Dataset preparation and baseline codec (Weeks 2–3)

- Download and validate the selected dataset; define reproducible splits.
- Implement preprocessing and patch extraction.
- Integrate a CompressAI baseline and implement training/validation loops.
- Track experiments in MLflow; evaluate PSNR, MS-SSIM and BPP.
- Verify actual encoding and decoding.

Deliverables: Dataset pipeline, functional baseline codec, initial checkpoints, baseline metrics and reproducible experiment.

Phase 3 — Saliency estimation and integration (Weeks 4–5)

- Integrate a pretrained saliency estimation model and generate maps.
- Visualize and validate saliency predictions.
- Implement saliency-weighted distortion and align maps with latent dimensions.
- Add saliency-conditioned latent modulation.
- Compare against the baseline and track experiments.

Deliverables: Saliency module, map generation pipeline, saliency-weighted baseline, initial saliency-conditioned codec and comparison report.

Phase 4 — Adaptive latent allocation (Weeks 6–7)

- Implement a spatial allocation mechanism.
- Investigate adaptive quantization or latent gating.

- Ensure allocation is compatible with the entropy model.
- Verify encoder/decoder synchronization and implement real bitstream serialization.
- Include metadata and side information in bitrate calculations.
- Evaluate allocation patterns, ROI fidelity and matched-bitrate performance.

Deliverables: Adaptive allocation module, functional compressed bitstream, actual BPP measurement, allocation visualizations and matched-bitrate evaluation.

Phase 5 — Perceptual and generative enhancement (Weeks 8–9)

- Integrate LPIPS-based perceptual supervision and evaluate reconstruction quality.
- Tune rate-distortion trade-offs.
- Investigate a PatchGAN discriminator and compare adversarial/non-adversarial variants.
- Measure training stability and computational overhead; retain extensions only when they demonstrate value.

Deliverables: Perceptual training configuration, optional adversarial model, perceptual evaluation results and ablation report.

Phase 6 — Final evaluation and release (Week 10)

- Run final experiments using fixed configurations.
- Evaluate selected baselines, rate-distortion curves and ROI metrics.
- Perform ablations and benchmark encoding/decoding performance.
- Validate reproducibility and prepare final documentation and demonstration.
- Tag a stable GitHub release.

Deliverables: Final checkpoints, evaluation results, comparison tables and graphs, technical report, reproducible code/configurations and optional demo.

Milestone 1: Baseline codec working. **Milestone 2:** Saliency integration complete. **Milestone 3:** Core research contribution implemented.

The GAN component is an extension, not a dependency for completing the core project. If the adaptive codec is not stable by the end of Week 7, prioritize bitstream correctness and final evaluation over GAN implementation.

13. Computational Requirements

The project will use a GPU-oriented training workflow while keeping preprocessing, evaluation utilities, testing and much of the data pipeline CPU-compatible.

Component	Recommended
GPU	NVIDIA GPU with CUDA support
VRAM	6 GB minimum for constrained experiments; 8–12 GB preferred
System RAM	16–32 GB
Storage	100–200 GB free, depending on dataset and checkpoints
CPU	Modern multi-core processor
OS	Windows with WSL2 Ubuntu or native Linux

The available RTX 4050 Laptop GPU has approximately 6 GB of usable VRAM, so the initial architecture should be designed with memory efficiency in mind.

Initial training configuration

Parameter	Starting value
Image patch size	256 × 256
Batch size	2
Precision	Mixed precision (FP16/BF16 where supported)
Optimizer	Adam
Learning rate	0.0001
Gradient accumulation	2
Data loader workers	4–6
Checkpoint frequency	Every 1–2 epochs

These are starting configurations, not guaranteed optimal values. Perform a short pilot run before committing to long training schedules.

GPU monitoring

```
nvidia-smi -l 2
```

```
torch.cuda.reset_peak_memory_stats()

# Run training steps here

allocated = torch.cuda.memory_allocated() / 1024**3
reserved = torch.cuda.memory_reserved() / 1024**3
peak = torch.cuda.max_memory_allocated() / 1024**3

print(f"Allocated: {allocated:.2f} GB")
print(f"Reserved:  {reserved:.2f} GB")
print(f"Peak:      {peak:.2f} GB")
```

Profile approximately 500 steps and estimate total training time from measured step time. This is more reliable than planning around theoretical estimates.

14. Expected Deliverables

Category	Deliverable
Research	Literature review and research specification
Dataset	Reproducible dataset preparation pipeline
Baseline	Functional neural image compression model
Saliency	Saliency estimation and integration module
Core model	Saliency-conditioned adaptive allocation
Compression	Quantization, entropy coding and actual bitstream
Decoder	Standalone reconstruction from compressed bitstream
Training	Configurable and reproducible training pipeline
MLOps	MLflow experiment tracking
Data management	DVC dataset and pipeline versioning
Development	GitHub repository with branch and PR workflow
Evaluation	Benchmark and ablation results
Documentation	Architecture, setup and experiment documentation
Demonstration	Optional web interface for image compression

15. Success Criteria

Functional requirements

- The system accepts an input image and generates a compressed bitstream.
- The bitstream can be decoded without requiring the original image.
- The reconstructed image has the same dimensions as the input.
- Actual bitrate includes the complete bitstream and necessary overhead.
- The encoder and decoder operate consistently.

Research requirements

- Compare the proposed model with conventional and learned compression baselines.
- Include matched-bitrate comparisons.
- Measure salient-region quality independently from background quality.
- Isolate adaptive allocation through ablation studies.
- Demonstrate a measurable improvement in at least one predefined perceptual or ROI-oriented metric without unacceptable degradation in overall rate-distortion performance.

Fix the precise numerical improvement target after baseline measurements; do not promise a percentage improvement before conducting experiments.

Engineering requirements

- All major experiments are logged in MLflow.
- Dataset and preprocessing versions are managed through DVC.

- GitHub contains a documented development history.
- Code changes are merged through reviewed pull requests.
- Automated tests validate important modules and the encode/decode pipeline.
- Training and evaluation configurations are reproducible.

16. Risks and Mitigation Strategy

Risk	Potential impact	Mitigation
GPU memory limitations	Training crashes or slow experiments	Smaller patches, mixed precision, gradient accumulation
Saliency prediction errors	Important regions receive inappropriate priority	Validate maps and test estimators
Entropy coding complexity	Estimated bitrate differs from actual bitrate	Implement and test real bitstream encoding early
Decoder incompatibility	Bitstream cannot be reconstructed independently	Establish encoder/decoder contracts and round-trip tests
Poor rate-distortion performance	Proposed codec underperforms baseline	Establish a strong baseline and tune progressively
GAN instability	Training divergence and poor reconstruction	Keep GAN optional; introduce after stable training
Dataset leakage	Misleading benchmark results	Fix splits and isolate final test data
Excessive experimentation	Project deadline missed	Set experiment budgets and prioritize core objectives
Weak research novelty	Limited academic contribution	Focus on measured adaptive allocation improvements

17. Research Novelty and Expected Contribution

The project is based on established research areas, including learned image compression, saliency estimation, perceptual loss functions and generative reconstruction. These individual techniques are not new by themselves.

Research question: Does explicitly conditioning latent coding allocation on visual saliency improve important-region reconstruction quality at matched overall bitrates, compared with a standard learned codec and a saliency-weighted distortion-only approach?

- An end-to-end learned image codec with saliency-conditioned latent allocation.
- A practical implementation of adaptive compression with actual entropy-coded bitstreams.
- An experimental comparison between saliency-weighted distortion and explicit adaptive allocation.
- An evaluation framework for measuring perceptual quality and ROI preservation at matched bitrates.
- An ablation-based analysis of saliency, adaptive allocation and perceptual reconstruction objectives.

Do not claim that combining saliency maps, GANs and neural compression is itself novel. The research value must come from the specific allocation mechanism, its implementation and experimental evidence.

18. Project Management and Reporting Schedule

Frequency	Activity	Expected output
Daily	Update active GitHub issues	Current task status
Per feature	Pull request and code review	Reviewed code
Per experiment	MLflow logging	Reproducible experiment record
Weekly	Review roadmap and blockers	Updated project board
Weekly	Review experimental results	Short experiment report
Per milestone	Integration and testing	Verified milestone
Final week	Release and documentation	Final project package

A weekly experiment report should contain: what changed; why the experiment was conducted; configuration and Git commit; dataset revision; training and validation metrics; GPU utilization and memory; observations and conclusions; and the next experiment.

19. Final Project Scope and Priorities

Priority	Scope
Core — Required	Learned neural compression baseline; saliency estimation; saliency-conditioned latent allocation; quantization and entropy modelling; actual compressed bitstream; independent decoder; MLflow and DVC; benchmarking and ablations.
Advanced — Recommended if feasible	LPIPS perceptual objective; adaptive quantization; advanced entropy modelling; latent allocation visualizations; Optuna optimization.
Optional — Stretch goal	PatchGAN adversarial training; Swin Transformer encoder; Streamlit demonstration; advanced saliency estimation; additional generalization experiments.

Prioritize the core scope before optional extensions. A reliable adaptive neural codec with real compression and rigorous evaluation is a stronger outcome than a complex generative architecture that cannot demonstrate measurable compression improvements.

20. Conclusion

The proposed project aims to develop a saliency-aware neural image compression system that moves beyond uniform compression by incorporating perceptual importance into the learned coding process.

It will combine neural feature extraction, saliency estimation, adaptive latent allocation, entropy modelling and generative reconstruction in a reproducible end-to-end pipeline.

The project will follow professional machine learning engineering practices through GitHub-based development, pull-request reviews, MLflow experiment tracking, DVC data versioning, automated testing and structured project management.

The final outcome will be a functional and experimentally evaluated neural compression system, supported by reproducible code, trained models, benchmark results and technical documentation.

21. Project Acceptance Checklist

- Research question and evaluation protocol finalized.
- GitHub repository and branch protection configured.
- MLflow experiment tracking operational.

- DVC configured and dataset versioned.
- Baseline codec trains and reconstructs images.
- Actual bitstream encoding and decoding verified.
- Saliency estimator integrated.
- Saliency-conditioned allocation implemented.
- Matched-bitrate evaluation completed.
- ROI and perceptual metrics evaluated.
- Ablation studies completed.
- Final documentation and reproducibility checks completed.

Final recommendation: Treat adaptive latent allocation and the actual entropy-coded bitstream as the central deliverables. Keep the generative adversarial component optional. This gives the project a clearer research contribution, a measurable outcome and a realistic path to completion.

Prepared as a technical project proposal and implementation roadmap. Dataset, architecture and training choices should be finalized through baseline experiments and documented research decisions.